

Consultas de prueba

Validaciones antes de insertar desde **staging** hacia **transformation**

```
-- Validación del campo: order_id

-- Total de registros
SELECT COUNT(*) AS total_registros
FROM staging.orders;

-- Valores vacíos o nulos
SELECT COUNT(*) AS order_id_vacios_o_nulos
FROM staging.orders
WHERE NULLIF(TRIM(order_id), '') IS NULL;

-- Valores con espacios externos
SELECT COUNT(*) AS order_id_con_espacios
FROM staging.orders
WHERE order_id <> TRIM(order_id);

-- Cantidad de order_id únicos
SELECT COUNT(DISTINCT NULLIF(TRIM(order_id), '')) AS order_id
_unicos
FROM staging.orders;

-- Cantidad de order_id duplicados
SELECT COUNT(*) AS order_id_duplicados
FROM (
    SELECT NULLIF(TRIM(order_id), '') AS order_id_limpio
    FROM staging.orders
    GROUP BY NULLIF(TRIM(order_id), '')
    HAVING COUNT(*) > 1
) AS duplicados;
```

```

-- Muestra de order_id duplicados
SELECT
    NULLIF(TRIM(order_id), '') AS order_id_limpio,
    COUNT(*) AS repeticiones
FROM staging.orders
GROUP BY NULLIF(TRIM(order_id), '')
HAVING COUNT(*) > 1
ORDER BY repeticiones DESC, order_id_limpio
LIMIT 20;

-- Muestra de transformación
SELECT
    order_id AS order_id_original,
    NULLIF(TRIM(order_id), '') AS order_id_transformado
FROM staging.orders
LIMIT 20;

```

```

-- Validación del campo: order_date

-- Valores vacíos o nulos
SELECT COUNT(*) AS order_date_vacias_o_nulas
FROM staging.orders
WHERE NULLIF(TRIM(order_date), '') IS NULL;

-- Fechas con diagonal
SELECT COUNT(*) AS order_date_con_diagonal
FROM staging.orders
WHERE order_date LIKE '%/%';

-- Fechas con guion
SELECT COUNT(*) AS order_date_con_guion
FROM staging.orders
WHERE order_date LIKE '%-%';

```

```
-- Valores con formato distinto a DD/MM/YYYY o DD-MM-YYYY
SELECT DISTINCT order_date
FROM staging.orders
WHERE NULLIF(TRIM(order_date), '') IS NOT NULL
      AND REPLACE(TRIM(order_date), '-', '/') !~ '^[0-9]{1,2}/[0-9]{1,2}/[0-9]{4}$'
LIMIT 20;
```

```
-- Probar conversión
SELECT
    order_date AS order_date_original,
    REPLACE(TRIM(order_date), '-', '/') AS order_date_normali
zada,
    CASE
        WHEN NULLIF(TRIM(order_date), '') IS NULL THEN NULL
        ELSE TO_DATE(REPLACE(TRIM(order_date), '-', '/'), 'D
D/MM/YYYY')
    END AS order_date_convertida
FROM staging.orders
LIMIT 20;
```

```
-- Rango de fechas
SELECT
    MIN(
        CASE
            WHEN NULLIF(TRIM(order_date), '') IS NULL THEN NU
LL
            ELSE TO_DATE(REPLACE(TRIM(order_date), '-', '/'),
'DD/MM/YYYY')
        END
    ) AS fecha_minima,
    MAX(
        CASE
            WHEN NULLIF(TRIM(order_date), '') IS NULL THEN NU
LL
```

```

        ELSE TO_DATE(REPLACE(TRIM(order_date), '-', '/'),
'DD/MM/YYYY')
    END
) AS fecha_maxima
FROM staging.orders;

-- Coherencia entre order_date y order_year
SELECT COUNT(*) AS inconsistencias_order_year
FROM staging.orders
WHERE NULLIF(TRIM(order_date), '') IS NOT NULL
    AND NULLIF(TRIM(order_year), '') IS NOT NULL
    AND EXTRACT(YEAR FROM TO_DATE(REPLACE(TRIM(order_date), '-',
', /'), 'DD/MM/YYYY'))::INTEGER
    <> NULLIF(REPLACE(TRIM(order_year), ',', ''), '')::INTE
GER;

```

```

-- Validación del campo: ship_date

-- Valores vacíos o nulos
SELECT COUNT(*) AS ship_date_vacias_o_nulas
FROM staging.orders
WHERE NULLIF(TRIM(ship_date), '') IS NULL;

-- Fechas con diagonal
SELECT COUNT(*) AS ship_date_con_diagonal
FROM staging.orders
WHERE ship_date LIKE '%/%';

-- Fechas con guion
SELECT COUNT(*) AS ship_date_con_guion
FROM staging.orders
WHERE ship_date LIKE '%-%';

-- Valores con formato distinto a DD/MM/YYYY o DD-MM-YYYY

```

```

SELECT DISTINCT ship_date
FROM staging.orders
WHERE NULLIF(TRIM(ship_date), '') IS NOT NULL
      AND REPLACE(TRIM(ship_date), '-', '/') !~ '^[0-9]{1,2}/[0-9]{1,2}/[0-9]{4}$'
LIMIT 20;

-- Probar conversión
SELECT
    ship_date AS ship_date_original,
    REPLACE(TRIM(ship_date), '-', '/') AS ship_date_normaliza
da,
    CASE
        WHEN NULLIF(TRIM(ship_date), '') IS NULL THEN NULL
        ELSE TO_DATE(REPLACE(TRIM(ship_date), '-', '/'), 'DD/
MM/YYYY')
    END AS ship_date_convertida
FROM staging.orders
LIMIT 20;

-- Rango de fechas de envío
SELECT
    MIN(
        CASE
            WHEN NULLIF(TRIM(ship_date), '') IS NULL THEN NUL
L
            ELSE TO_DATE(REPLACE(TRIM(ship_date), '-', '/'),
'DD/MM/YYYY')
        END
    ) AS ship_date_minima,
    MAX(
        CASE
            WHEN NULLIF(TRIM(ship_date), '') IS NULL THEN NUL
L
            ELSE TO_DATE(REPLACE(TRIM(ship_date), '-', '/'),
'DD/MM/YYYY')

```

```

        END
    ) AS ship_date_maxima
FROM staging.orders;

-- Revisar envíos anteriores a la fecha de orden
SELECT COUNT(*) AS envios_antes_de_orden
FROM staging.orders
WHERE NULLIF(TRIM(order_date), '') IS NOT NULL
    AND NULLIF(TRIM(ship_date), '') IS NOT NULL
    AND TO_DATE(REPLACE(TRIM(ship_date), '-', '/'), 'DD/MM/YYYY')
    < TO_DATE(REPLACE(TRIM(order_date), '-', '/'), 'DD/MM/YYYY');

```

```

-- Validación del campo: ship_mode

-- Valores vacíos o nulos
SELECT COUNT(*) AS ship_mode_vacios_o_nulos
FROM staging.orders
WHERE NULLIF(TRIM(ship_mode), '') IS NULL;

-- Valores únicos originales
SELECT DISTINCT ship_mode
FROM staging.orders
ORDER BY ship_mode;

-- Valores únicos normalizados
SELECT DISTINCT UPPER(TRIM(ship_mode)) AS ship_mode_normalizado
FROM staging.orders
ORDER BY ship_mode_normalizado;

-- Valores con espacios externos
SELECT COUNT(*) AS ship_mode_con_espacios

```

```

FROM staging.orders
WHERE ship_mode <> TRIM(ship_mode);

-- Frecuencia por modo de envío
SELECT
    UPPER(TRIM(ship_mode)) AS ship_mode_normalizado,
    COUNT(*) AS total_registros
FROM staging.orders
GROUP BY UPPER(TRIM(ship_mode))
ORDER BY total_registros DESC;

-- Muestra de transformación
SELECT
    ship_mode AS ship_mode_original,
    NULLIF(UPPER(TRIM(ship_mode)), '') AS ship_mode_transformado
FROM staging.orders
LIMIT 20;

```

```

-- Validación del campo: customer_name

-- Valores vacíos o nulos
SELECT COUNT(*) AS customer_name_vacios_o_nulos
FROM staging.orders
WHERE NULLIF(TRIM(customer_name), '') IS NULL;

-- Valores con espacios externos
SELECT COUNT(*) AS customer_name_con_espacios
FROM staging.orders
WHERE customer_name <> TRIM(customer_name);

-- Ejemplos con espacios externos
SELECT

```

```

        customer_name AS customer_name_original,
        TRIM(customer_name) AS customer_name_limpio
FROM staging.orders
WHERE customer_name <> TRIM(customer_name)
LIMIT 20;

-- Cantidad de clientes distintos
SELECT COUNT(DISTINCT NULLIF(TRIM(customer_name), '')) AS cli
entes_distintos
FROM staging.orders;

-- Nombres repetidos
SELECT
    NULLIF(TRIM(customer_name), '') AS customer_name_limpio,
    COUNT(*) AS repeticiones
FROM staging.orders
GROUP BY NULLIF(TRIM(customer_name), '')
HAVING COUNT(*) > 1
ORDER BY repeticiones DESC, customer_name_limpio
LIMIT 20;

-- Longitud mínima, máxima y promedio
SELECT
    MIN(LENGTH(TRIM(customer_name))) AS longitud_minima,
    MAX(LENGTH(TRIM(customer_name))) AS longitud_maxima,
    AVG(LENGTH(TRIM(customer_name))) AS longitud_promedio
FROM staging.orders
WHERE NULLIF(TRIM(customer_name), '') IS NOT NULL;

-- Muestra de transformación
SELECT
    customer_name AS customer_name_original,
    NULLIF(TRIM(customer_name), '') AS customer_name_transfor
mado

```

```
FROM staging.orders  
LIMIT 20;
```

```
-- Validación del campo: segment  
  
-- Valores vacíos o nulos  
SELECT COUNT(*) AS segment_vacios_o_nulos  
FROM staging.orders  
WHERE NULLIF(TRIM(segment), '') IS NULL;  
  
-- Valores únicos originales  
SELECT DISTINCT segment  
FROM staging.orders  
ORDER BY segment;  
  
-- Valores únicos normalizados  
SELECT DISTINCT UPPER(TRIM(segment)) AS segment_normalizado  
FROM staging.orders  
ORDER BY segment_normalizado;  
  
-- Valores con espacios externos  
SELECT COUNT(*) AS segment_con_espacios  
FROM staging.orders  
WHERE segment <> TRIM(segment);  
  
-- Frecuencia por segmento  
SELECT  
    UPPER(TRIM(segment)) AS segment_normalizado,  
    COUNT(*) AS total_registros  
FROM staging.orders  
GROUP BY UPPER(TRIM(segment))  
ORDER BY total_registros DESC;  
  
-- Muestra de transformación
```

```

SELECT
    segment AS segment_original,
    NULLIF(UPPER(TRIM(segment)), '') AS segment_transformado
FROM staging.orders
LIMIT 20;

```

```

-- Validación del campo: state_name

-- Valores vacíos o nulos
SELECT COUNT(*) AS state_name_vacios_o_nulos
FROM staging.orders
WHERE NULLIF(TRIM(state_name), '') IS NULL;

-- Valores únicos originales
SELECT DISTINCT state_name
FROM staging.orders
ORDER BY state_name;

-- Valores únicos normalizados
SELECT DISTINCT UPPER(TRIM(state_name)) AS state_name_normali
zado
FROM staging.orders
ORDER BY state_name_normalizado;

-- Valores con espacios externos
SELECT COUNT(*) AS state_name_con_espacios
FROM staging.orders
WHERE state_name <> TRIM(state_name);

-- Frecuencia por estado/provincia
SELECT
    UPPER(TRIM(state_name)) AS state_name_normalizado,
    COUNT(*) AS total_registros
FROM staging.orders

```

```

GROUP BY UPPER(TRIM(state_name))
ORDER BY total_registros DESC
LIMIT 50;

-- Relación estado-país
SELECT
    UPPER(TRIM(country)) AS country_normalizado,
    UPPER(TRIM(state_name)) AS state_name_normalizado,
    COUNT(*) AS total_registros
FROM staging.orders
GROUP BY
    UPPER(TRIM(country)),
    UPPER(TRIM(state_name))
ORDER BY total_registros DESC
LIMIT 50;

-- Muestra de transformación
SELECT
    state_name AS state_name_original,
    NULLIF(UPPER(TRIM(state_name)), '') AS state_name_transfo
rmando
FROM staging.orders
LIMIT 20;

```

```

-- Validación del campo: country

-- Valores vacíos o nulos
SELECT COUNT(*) AS country_vacios_o_nulos
FROM staging.orders
WHERE NULLIF(TRIM(country), '') IS NULL;

-- Valores únicos originales
SELECT DISTINCT country
FROM staging.orders

```

```

ORDER BY country;

-- Valores únicos normalizados
SELECT DISTINCT UPPER(TRIM(country)) AS country_normalizado
FROM staging.orders
ORDER BY country_normalizado;

-- Valores con espacios externos
SELECT COUNT(*) AS country_con_espacios
FROM staging.orders
WHERE country <> TRIM(country);

-- Frecuencia por país
SELECT
    UPPER(TRIM(country)) AS country_normalizado,
    COUNT(*) AS total_registros
FROM staging.orders
GROUP BY UPPER(TRIM(country))
ORDER BY total_registros DESC;

-- Relación país, mercado y región
SELECT
    UPPER(TRIM(country)) AS country_normalizado,
    UPPER(TRIM(market)) AS market_normalizado,
    UPPER(TRIM(region)) AS region_normalizado,
    COUNT(*) AS total_registros
FROM staging.orders
GROUP BY
    UPPER(TRIM(country)),
    UPPER(TRIM(market)),
    UPPER(TRIM(region))
ORDER BY total_registros DESC
LIMIT 50;

-- Muestra de transformación
SELECT

```

```
country AS country_original,  
    NULLIF(UPPER(TRIM(country)), '') AS country_transformado  
FROM staging.orders  
LIMIT 20;
```

```
-- Validación del campo: market
```

```
-- Valores vacíos o nulos
```

```
SELECT COUNT(*) AS market_vacios_o_nulos  
FROM staging.orders  
WHERE NULLIF(TRIM(market), '') IS NULL;
```

```
-- Valores únicos originales
```

```
SELECT DISTINCT market  
FROM staging.orders  
ORDER BY market;
```

```
-- Valores únicos normalizados
```

```
SELECT DISTINCT UPPER(TRIM(market)) AS market_normalizado  
FROM staging.orders  
ORDER BY market_normalizado;
```

```
-- Valores con espacios externos
```

```
SELECT COUNT(*) AS market_con_espacios  
FROM staging.orders  
WHERE market <> TRIM(market);
```

```
-- Frecuencia por mercado
```

```
SELECT  
    UPPER(TRIM(market)) AS market_normalizado,  
    COUNT(*) AS total_registros  
FROM staging.orders  
GROUP BY UPPER(TRIM(market))
```

```

ORDER BY total_registros DESC;

-- Relación mercado-país
SELECT
    UPPER(TRIM(market)) AS market_normalizado,
    UPPER(TRIM(country)) AS country_normalizado,
    COUNT(*) AS total_registros
FROM staging.orders
GROUP BY
    UPPER(TRIM(market)),
    UPPER(TRIM(country))
ORDER BY total_registros DESC
LIMIT 50;

-- Muestra de transformación
SELECT
    market AS market_original,
    NULLIF(UPPER(TRIM(market)), '') AS market_transformado
FROM staging.orders
LIMIT 20;

```

```

-- Validación del campo: region

-- Valores vacíos o nulos
SELECT COUNT(*) AS region_vacios_o_nulos
FROM staging.orders
WHERE NULLIF(TRIM(region), '') IS NULL;

-- Valores únicos originales
SELECT DISTINCT region
FROM staging.orders
ORDER BY region;

-- Valores únicos normalizados

```

```

SELECT DISTINCT UPPER(TRIM(region)) AS region_normalizada
FROM staging.orders
ORDER BY region_normalizada;

-- Valores con espacios externos
SELECT COUNT(*) AS region_con_espacios
FROM staging.orders
WHERE region <> TRIM(region);

-- Frecuencia por región
SELECT
    UPPER(TRIM(region)) AS region_normalizada,
    COUNT(*) AS total_registros
FROM staging.orders
GROUP BY UPPER(TRIM(region))
ORDER BY total_registros DESC;

-- Relación región, mercado y país
SELECT
    UPPER(TRIM(region)) AS region_normalizada,
    UPPER(TRIM(market)) AS market_normalizado,
    UPPER(TRIM(country)) AS country_normalizado,
    COUNT(*) AS total_registros
FROM staging.orders
GROUP BY
    UPPER(TRIM(region)),
    UPPER(TRIM(market)),
    UPPER(TRIM(country))
ORDER BY total_registros DESC
LIMIT 50;

-- Muestra de transformación
SELECT
    region AS region_original,
    NULLIF(UPPER(TRIM(region)), '') AS region_transformada

```

```
FROM staging.orders  
LIMIT 20;
```

```
-- Validación del campo: product_id
```

```
-- Valores vacíos o nulos
```

```
SELECT COUNT(*) AS product_id_vacios_o_nulos  
FROM staging.orders  
WHERE NULLIF(TRIM(product_id), '') IS NULL;
```

```
-- Valores con espacios externos
```

```
SELECT COUNT(*) AS product_id_con_espacios  
FROM staging.orders  
WHERE product_id <> TRIM(product_id);
```

```
-- Cantidad de product_id únicos
```

```
SELECT COUNT(DISTINCT NULLIF(TRIM(product_id), '')) AS produc  
t_id_unicos  
FROM staging.orders;
```

```
-- Frecuencia de productos
```

```
SELECT  
    NULLIF(TRIM(product_id), '') AS product_id_limpio,  
    COUNT(*) AS repeticiones  
FROM staging.orders  
GROUP BY NULLIF(TRIM(product_id), '')  
ORDER BY repeticiones DESC, product_id_limpio  
LIMIT 50;
```

```
-- Revisar relación product_id y product_name
```

```
SELECT  
    NULLIF(TRIM(product_id), '') AS product_id_limpio,  
    COUNT(DISTINCT NULLIF(TRIM(product_name), '')) AS nombres
```

```

_distintos
FROM staging.orders
GROUP BY NULLIF(TRIM(product_id), '')
HAVING COUNT(DISTINCT NULLIF(TRIM(product_name), '')) > 1
ORDER BY nombres_distintos DESC
LIMIT 20;

-- Muestra de transformación
SELECT
    product_id AS product_id_original,
    NULLIF(TRIM(product_id), '') AS product_id_transformado
FROM staging.orders
LIMIT 20;

```

```

-- Validación del campo: category

-- Valores vacíos o nulos
SELECT COUNT(*) AS category_vacios_o_nulos
FROM staging.orders
WHERE NULLIF(TRIM(category), '') IS NULL;

-- Valores únicos originales
SELECT DISTINCT category
FROM staging.orders
ORDER BY category;

-- Valores únicos normalizados
SELECT DISTINCT UPPER(TRIM(category)) AS category_normalizada
FROM staging.orders
ORDER BY category_normalizada;

-- Valores con espacios externos
SELECT COUNT(*) AS category_con_espacios

```

```

FROM staging.orders
WHERE category <> TRIM(category);

-- Frecuencia por categoría
SELECT
    UPPER(TRIM(category)) AS category_normalizada,
    COUNT(*) AS total_registros
FROM staging.orders
GROUP BY UPPER(TRIM(category))
ORDER BY total_registros DESC;

-- Relación categoría-subcategoría
SELECT
    UPPER(TRIM(category)) AS category_normalizada,
    UPPER(TRIM(sub_category)) AS sub_category_normalizada,
    COUNT(*) AS total_registros
FROM staging.orders
GROUP BY
    UPPER(TRIM(category)),
    UPPER(TRIM(sub_category))
ORDER BY category_normalizada, total_registros DESC;

-- Muestra de transformación
SELECT
    category AS category_original,
    NULLIF(UPPER(TRIM(category)), '') AS category_transformad
a
FROM staging.orders
LIMIT 20;

```

```

-- Validación del campo: sub_category

```

```

-- Valores vacíos o nulos

```

```

SELECT COUNT(*) AS sub_category_vacios_o_nulos

```

```

FROM staging.orders
WHERE NULLIF(TRIM(sub_category), '') IS NULL;

-- Valores únicos originales
SELECT DISTINCT sub_category
FROM staging.orders
ORDER BY sub_category;

-- Valores únicos normalizados
SELECT DISTINCT UPPER(TRIM(sub_category)) AS sub_category_normalizada
FROM staging.orders
ORDER BY sub_category_normalizada;

-- Valores con espacios externos
SELECT COUNT(*) AS sub_category_con_espacios
FROM staging.orders
WHERE sub_category <> TRIM(sub_category);

-- Frecuencia por subcategoría
SELECT
    UPPER(TRIM(sub_category)) AS sub_category_normalizada,
    COUNT(*) AS total_registros
FROM staging.orders
GROUP BY UPPER(TRIM(sub_category))
ORDER BY total_registros DESC;

-- Relación subcategoría-categoría
SELECT
    UPPER(TRIM(sub_category)) AS sub_category_normalizada,
    COUNT(DISTINCT UPPER(TRIM(category))) AS categorias_distintas
FROM staging.orders
GROUP BY UPPER(TRIM(sub_category))
HAVING COUNT(DISTINCT UPPER(TRIM(category))) > 1
ORDER BY categorias_distintas DESC;

```

```
-- Muestra de transformación
SELECT
    sub_category AS sub_category_original,
    NULLIF(UPPER(TRIM(sub_category)), '') AS sub_category_tra
nsformada
FROM staging.orders
LIMIT 20;
```

```
-- Validación del campo: product_name

-- Valores vacíos o nulos
SELECT COUNT(*) AS product_name_vacios_o_nulos
FROM staging.orders
WHERE NULLIF(TRIM(product_name), '') IS NULL;

-- Valores con espacios externos
SELECT COUNT(*) AS product_name_con_espacios
FROM staging.orders
WHERE product_name <> TRIM(product_name);

-- Cantidad de productos distintos por nombre
SELECT COUNT(DISTINCT NULLIF(TRIM(product_name), '')) AS prod
uct_name_unicos
FROM staging.orders;

-- Longitud mínima, máxima y promedio
SELECT
    MIN(LENGTH(TRIM(product_name))) AS longitud_minima,
    MAX(LENGTH(TRIM(product_name))) AS longitud_maxima,
    AVG(LENGTH(TRIM(product_name))) AS longitud_promedio
FROM staging.orders
WHERE NULLIF(TRIM(product_name), '') IS NOT NULL;
```

```

-- Nombres de producto más largos
SELECT
    product_name,
    LENGTH(TRIM(product_name)) AS longitud
FROM staging.orders
WHERE NULLIF(TRIM(product_name), '') IS NOT NULL
ORDER BY longitud DESC
LIMIT 20;

-- Productos repetidos
SELECT
    NULLIF(TRIM(product_name), '') AS product_name_limpio,
    COUNT(*) AS repeticiones
FROM staging.orders
GROUP BY NULLIF(TRIM(product_name), '')
ORDER BY repeticiones DESC
LIMIT 50;

-- Muestra de transformación
SELECT
    product_name AS product_name_original,
    NULLIF(TRIM(product_name), '') AS product_name_transformado
FROM staging.orders
LIMIT 20;

```

```

-- Validación del campo: sales

-- Valores vacíos o nulos
SELECT COUNT(*) AS sales_vacios_o_nulos
FROM staging.orders
WHERE NULLIF(REPLACE(TRIM(sales), ',', ''), '') IS NULL;

-- Valores con coma

```

```

SELECT COUNT(*) AS sales_con_comas
FROM staging.orders
WHERE sales LIKE '%,%';

-- Valores no numéricos después de quitar comas
SELECT sales
FROM staging.orders
WHERE NULLIF(REPLACE(TRIM(sales), ',', ''), '') IS NOT NULL
  AND REPLACE(TRIM(sales), ',', '') !~ '^-[0-9]+(\.[0-9]+)?$'
LIMIT 20;

-- Valores con decimales
SELECT COUNT(*) AS sales_con_decimales
FROM staging.orders
WHERE NULLIF(REPLACE(TRIM(sales), ',', ''), '') IS NOT NULL
  AND REPLACE(TRIM(sales), ',', '') ~ '^-[0-9]+\.[0-9]+$';

-- Valores negativos
SELECT COUNT(*) AS sales_negativos
FROM staging.orders
WHERE NULLIF(REPLACE(TRIM(sales), ',', ''), '') IS NOT NULL
  AND REPLACE(TRIM(sales), ',', '')::NUMERIC < 0;

-- Rango de valores
SELECT
  MIN(REPLACE(TRIM(sales), ',', '')::NUMERIC) AS sales_minimo,
  MAX(REPLACE(TRIM(sales), ',', '')::NUMERIC) AS sales_maximo,
  AVG(REPLACE(TRIM(sales), ',', '')::NUMERIC) AS sales_promedio
FROM staging.orders
WHERE NULLIF(REPLACE(TRIM(sales), ',', ''), '') IS NOT NULL
  AND REPLACE(TRIM(sales), ',', '') ~ '^-[0-9]+(\.[0-9]+)?$';

```

```
-- Muestra de transformación
SELECT
    sales AS sales_original,
    REPLACE(TRIM(sales), ',', '') AS sales_sin_comas,
    NULLIF(REPLACE(TRIM(sales), ',', ''), '')::NUMERIC AS sales_transformado
FROM staging.orders
LIMIT 20;
```

```
-- Validación del campo: quantity

-- Valores vacíos o nulos
SELECT COUNT(*) AS quantity_vacios_o_nulos
FROM staging.orders
WHERE NULLIF(REPLACE(TRIM(quantity), ',', ''), '') IS NULL;

-- Valores con coma
SELECT COUNT(*) AS quantity_con_comas
FROM staging.orders
WHERE quantity LIKE '%,%';

-- Valores no enteros
SELECT quantity
FROM staging.orders
WHERE NULLIF(REPLACE(TRIM(quantity), ',', ''), '') IS NOT NULL
    AND REPLACE(TRIM(quantity), ',', '') !~ '^-[0-9]+$'
LIMIT 20;

-- Valores negativos o cero
SELECT COUNT(*) AS quantity_menor_o_igual_cero
FROM staging.orders
```

```

WHERE NULLIF(REPLACE(TRIM(quantity), ',', ''), '') IS NOT NULL
AND REPLACE(TRIM(quantity), ',', '') ~ '^-?[0-9]+$'
AND REPLACE(TRIM(quantity), ',', '')::INTEGER <= 0;

-- Rango de valores
SELECT
    MIN(REPLACE(TRIM(quantity), ',', '')::INTEGER) AS quantity_minima,
    MAX(REPLACE(TRIM(quantity), ',', '')::INTEGER) AS quantity_maxima,
    AVG(REPLACE(TRIM(quantity), ',', '')::INTEGER) AS quantity_promedio
FROM staging.orders
WHERE NULLIF(REPLACE(TRIM(quantity), ',', ''), '') IS NOT NULL
AND REPLACE(TRIM(quantity), ',', '') ~ '^-?[0-9]+$';

-- Muestra de transformación
SELECT
    quantity AS quantity_original,
    NULLIF(REPLACE(TRIM(quantity), ',', ''), '')::INTEGER AS quantity_transformada
FROM staging.orders
LIMIT 20;

```

```

-- Validación del campo: discount

```

```

-- Valores vacíos o nulos

```

```

SELECT COUNT(*) AS discount_vacios_o_nulos
FROM staging.orders
WHERE NULLIF(REPLACE(TRIM(discount), ',', ''), '') IS NULL;

```

```

-- Valores con coma
SELECT COUNT(*) AS discount_con_comas
FROM staging.orders
WHERE discount LIKE '%,%';

-- Valores no numéricos
SELECT discount
FROM staging.orders
WHERE NULLIF(REPLACE(TRIM(discount), ',', ''), '') IS NOT NULL
    AND REPLACE(TRIM(discount), ',', '') !~ '^-[0-9]+(\.[0-9]+)?$'
LIMIT 20;

-- Valores negativos
SELECT COUNT(*) AS discount_negativos
FROM staging.orders
WHERE NULLIF(REPLACE(TRIM(discount), ',', ''), '') IS NOT NULL
    AND REPLACE(TRIM(discount), ',', '') ~ '^-[0-9]+(\.[0-9]+)?$'
    AND REPLACE(TRIM(discount), ',', '')::NUMERIC < 0;

-- Valores mayores a 1
SELECT COUNT(*) AS discount_mayores_a_1
FROM staging.orders
WHERE NULLIF(REPLACE(TRIM(discount), ',', ''), '') IS NOT NULL
    AND REPLACE(TRIM(discount), ',', '') ~ '^-[0-9]+(\.[0-9]+)?$'
    AND REPLACE(TRIM(discount), ',', '')::NUMERIC > 1;

-- Rango de valores
SELECT
    MIN(REPLACE(TRIM(discount), ',', '')::NUMERIC) AS discount_minimo,

```

```

        MAX(REPLACE(TRIM(discount), ',', ' '::NUMERIC) AS discount_maximo,
        AVG(REPLACE(TRIM(discount), ',', ' '::NUMERIC) AS discount_promedio
FROM staging.orders
WHERE NULLIF(REPLACE(TRIM(discount), ',', ' '), '') IS NOT NULL
        AND REPLACE(TRIM(discount), ',', ' ') ~ '^-[0-9]+(\.[0-9]+)?$';

```

```

-- Muestra de transformación
SELECT
    discount AS discount_original,
    NULLIF(REPLACE(TRIM(discount), ',', ' '), '')::NUMERIC AS discount_transformado
FROM staging.orders
LIMIT 20;

```

```

-- Validación del campo: profit

-- Valores vacíos o nulos
SELECT COUNT(*) AS profit_vacios_o_nulos
FROM staging.orders
WHERE NULLIF(REPLACE(TRIM(profit), ',', ' '), '') IS NULL;

-- Valores con coma
SELECT COUNT(*) AS profit_con_comas
FROM staging.orders
WHERE profit LIKE '%,%';

-- Valores no numéricos
SELECT profit
FROM staging.orders
WHERE NULLIF(REPLACE(TRIM(profit), ',', ' '), '') IS NOT NULL

```

```

    AND REPLACE(TRIM(profit), ',', '') !~ '^-?[0-9]+(\.[0-9]+)?
$'
LIMIT 20;

-- Valores negativos
SELECT COUNT(*) AS profit_negativos
FROM staging.orders
WHERE NULLIF(REPLACE(TRIM(profit), ',', ''), '') IS NOT NULL
    AND REPLACE(TRIM(profit), ',', '') ~ '^-?[0-9]+(\.[0-9]+)?
$'
    AND REPLACE(TRIM(profit), ',', '')::NUMERIC < 0;

-- Valores positivos
SELECT COUNT(*) AS profit_positivos
FROM staging.orders
WHERE NULLIF(REPLACE(TRIM(profit), ',', ''), '') IS NOT NULL
    AND REPLACE(TRIM(profit), ',', '') ~ '^-?[0-9]+(\.[0-9]+)?
$'
    AND REPLACE(TRIM(profit), ',', '')::NUMERIC > 0;

-- Valores en cero
SELECT COUNT(*) AS profit_cero
FROM staging.orders
WHERE NULLIF(REPLACE(TRIM(profit), ',', ''), '') IS NOT NULL
    AND REPLACE(TRIM(profit), ',', '') ~ '^-?[0-9]+(\.[0-9]+)?
$'
    AND REPLACE(TRIM(profit), ',', '')::NUMERIC = 0;

-- Rango de valores
SELECT
    MIN(REPLACE(TRIM(profit), ',', '')::NUMERIC) AS profit_mi
nimo,
    MAX(REPLACE(TRIM(profit), ',', '')::NUMERIC) AS profit_ma
ximo,
    AVG(REPLACE(TRIM(profit), ',', '')::NUMERIC) AS profit_pr
omedio

```

```

FROM staging.orders
WHERE NULLIF(REPLACE(TRIM(profit), ',', ''), '') IS NOT NULL
      AND REPLACE(TRIM(profit), ',', '') ~ '^-?[0-9]+(\.[0-9]+)?$';

-- Muestra de transformación
SELECT
    profit AS profit_original,
    NULLIF(REPLACE(TRIM(profit), ',', ''), '')::NUMERIC AS profit_transformado
FROM staging.orders
LIMIT 20;

```

```

-- Validación del campo: shipping_cost

-- Valores vacíos o nulos
SELECT COUNT(*) AS shipping_cost_vacios_o_nulos
FROM staging.orders
WHERE NULLIF(REPLACE(TRIM(shipping_cost), ',', ''), '') IS NULL;

-- Valores con coma
SELECT COUNT(*) AS shipping_cost_con_comas
FROM staging.orders
WHERE shipping_cost LIKE '%,%';

-- Valores no numéricos
SELECT shipping_cost
FROM staging.orders
WHERE NULLIF(REPLACE(TRIM(shipping_cost), ',', ''), '') IS NOT NULL
      AND REPLACE(TRIM(shipping_cost), ',', '') !~ '^-?[0-9]+(\.[0-9]+)?$'

```

```

LIMIT 20;

-- Valores negativos
SELECT COUNT(*) AS shipping_cost_negativos
FROM staging.orders
WHERE NULLIF(REPLACE(TRIM(shipping_cost), ',', ''), '') IS NOT NULL
    AND REPLACE(TRIM(shipping_cost), ',', '') ~ '^-[0-9]+(\.[0-9]+)?$'
    AND REPLACE(TRIM(shipping_cost), ',', '')::NUMERIC < 0;

-- Rango de valores
SELECT
    MIN(REPLACE(TRIM(shipping_cost), ',', '')::NUMERIC) AS shipping_cost_minimo,
    MAX(REPLACE(TRIM(shipping_cost), ',', '')::NUMERIC) AS shipping_cost_maximo,
    AVG(REPLACE(TRIM(shipping_cost), ',', '')::NUMERIC) AS shipping_cost_promedio
FROM staging.orders
WHERE NULLIF(REPLACE(TRIM(shipping_cost), ',', ''), '') IS NOT NULL
    AND REPLACE(TRIM(shipping_cost), ',', '') ~ '^-[0-9]+(\.[0-9]+)?$';

-- Muestra de transformación
SELECT
    shipping_cost AS shipping_cost_original,
    NULLIF(REPLACE(TRIM(shipping_cost), ',', ''), '')::NUMERIC AS shipping_cost_transformado
FROM staging.orders
LIMIT 20;

```

```

-- Validación del campo: order_priority

```

```

-- Valores vacíos o nulos
SELECT COUNT(*) AS order_priority_vacios_o_nulos
FROM staging.orders
WHERE NULLIF(TRIM(order_priority), '') IS NULL;

-- Valores únicos originales
SELECT DISTINCT order_priority
FROM staging.orders
ORDER BY order_priority;

-- Valores únicos normalizados
SELECT DISTINCT UPPER(TRIM(order_priority)) AS order_priority
_normalizado
FROM staging.orders
ORDER BY order_priority_normalizado;

-- Valores con espacios externos
SELECT COUNT(*) AS order_priority_con_espacios
FROM staging.orders
WHERE order_priority <> TRIM(order_priority);

-- Frecuencia por prioridad
SELECT
    UPPER(TRIM(order_priority)) AS order_priority_normalizad
o,
    COUNT(*) AS total_registros
FROM staging.orders
GROUP BY UPPER(TRIM(order_priority))
ORDER BY total_registros DESC;

-- Muestra de transformación
SELECT
    order_priority AS order_priority_original,
    NULLIF(UPPER(TRIM(order_priority)), '') AS order_priority
_transformado

```

```
FROM staging.orders  
LIMIT 20;
```

```
-- Validación del campo: order_year
```

```
-- Valores vacíos o nulos
```

```
SELECT COUNT(*) AS order_year_vacios_o_nulos  
FROM staging.orders  
WHERE NULLIF(REPLACE(TRIM(order_year), ',', ''), '') IS NULL;
```

```
-- Valores con coma
```

```
SELECT COUNT(*) AS order_year_con_comas  
FROM staging.orders  
WHERE order_year LIKE '%,%';
```

```
-- Valores no enteros
```

```
SELECT order_year  
FROM staging.orders  
WHERE NULLIF(REPLACE(TRIM(order_year), ',', ''), '') IS NOT NULL  
AND REPLACE(TRIM(order_year), ',', '') !~ '^[0-9]{4}$'  
LIMIT 20;
```

```
-- Valores únicos
```

```
SELECT DISTINCT NULLIF(REPLACE(TRIM(order_year), ',', ''),  
'')::INTEGER AS order_year_transformado  
FROM staging.orders  
WHERE NULLIF(REPLACE(TRIM(order_year), ',', ''), '') IS NOT NULL  
AND REPLACE(TRIM(order_year), ',', '') ~ '^[0-9]{4}$'  
ORDER BY order_year_transformado;
```

```
-- Rango de años
```

```
SELECT
```

```

        MIN(NULLIF(REPLACE(TRIM(order_year), ',', ''), '')::INTEGER) AS order_year_minimo,
        MAX(NULLIF(REPLACE(TRIM(order_year), ',', ''), '')::INTEGER) AS order_year_maximo
FROM staging.orders
WHERE NULLIF(REPLACE(TRIM(order_year), ',', ''), '') IS NOT NULL
    AND REPLACE(TRIM(order_year), ',', '') ~ '^[0-9]{4}$';

-- Coherencia con order_date
SELECT COUNT(*) AS inconsistencias_con_order_date
FROM staging.orders
WHERE NULLIF(TRIM(order_date), '') IS NOT NULL
    AND NULLIF(REPLACE(TRIM(order_year), ',', ''), '') IS NOT NULL
    AND REPLACE(TRIM(order_year), ',', '') ~ '^[0-9]{4}$'
    AND EXTRACT(YEAR FROM TO_DATE(REPLACE(TRIM(order_date), '-', '/'), 'DD/MM/YYYY'))::INTEGER
    <> NULLIF(REPLACE(TRIM(order_year), ',', ''), '')::INTEGER;

-- Muestra de transformación
SELECT
    order_year AS order_year_original,
    NULLIF(REPLACE(TRIM(order_year), ',', ''), '')::INTEGER AS order_year_transformado
FROM staging.orders
WHERE NULLIF(REPLACE(TRIM(order_year), ',', ''), '') IS NOT NULL
LIMIT 20;

```

Validaciones antes de insertar desde `transformation` hacia `analytics`

```
-- 1. Validar total de registros en transformation
SELECT COUNT(*) AS total_transformation
FROM transformation.orders;
```

```
-- 2. Revisar muestra de datos transformados
SELECT *
FROM transformation.orders
LIMIT 10;
```

```
-- 3. Validar registros con campos nulos
SELECT COUNT(*) AS registros_incompletos
FROM transformation.orders
WHERE order_id IS NULL
    OR order_date IS NULL
    OR ship_date IS NULL
    OR ship_mode IS NULL
    OR customer_name IS NULL
    OR segment IS NULL
    OR state_name IS NULL
    OR country IS NULL
    OR market IS NULL
    OR region IS NULL
    OR product_id IS NULL
    OR category IS NULL
    OR sub_category IS NULL
    OR product_name IS NULL
    OR sales IS NULL
    OR quantity IS NULL
    OR discount IS NULL
    OR profit IS NULL
    OR shipping_cost IS NULL
    OR order_priority IS NULL
    OR order_year IS NULL;
```

-- 4. Validar registros completos que pasarían a analytics

```
SELECT COUNT(*) AS registros_completos
```

```
FROM transformation.orders
```

```
WHERE order_id IS NOT NULL
```

```
    AND order_date IS NOT NULL
```

```
    AND ship_date IS NOT NULL
```

```
    AND ship_mode IS NOT NULL
```

```
    AND customer_name IS NOT NULL
```

```
    AND segment IS NOT NULL
```

```
    AND state_name IS NOT NULL
```

```
    AND country IS NOT NULL
```

```
    AND market IS NOT NULL
```

```
    AND region IS NOT NULL
```

```
    AND product_id IS NOT NULL
```

```
    AND category IS NOT NULL
```

```
    AND sub_category IS NOT NULL
```

```
    AND product_name IS NOT NULL
```

```
    AND sales IS NOT NULL
```

```
    AND quantity IS NOT NULL
```

```
    AND discount IS NOT NULL
```

```
    AND profit IS NOT NULL
```

```
    AND shipping_cost IS NOT NULL
```

```
    AND order_priority IS NOT NULL
```

```
    AND order_year IS NOT NULL;
```

-- 5. Comparar total de registros entre staging y transformation

```
SELECT
```

```
    (SELECT COUNT(*) FROM staging.orders) AS total_staging,
```

```
    (SELECT COUNT(*) FROM transformation.orders) AS total_transformation;
```

```

-- 6. Validar unicidad de record_id
SELECT
    COUNT(*) AS total_record_id,
    COUNT(DISTINCT record_id) AS record_id_unicos
FROM transformation.orders;

-- 7. Validar reglas numéricas antes de analytics
SELECT
    COUNT(*) FILTER (WHERE sales < 0) AS sales_negativos,
    COUNT(*) FILTER (WHERE quantity <= 0) AS quantity_cero_o_
negativa,
    COUNT(*) FILTER (WHERE discount < 0 OR discount > 1) AS d
iscount_fuera_rango,
    COUNT(*) FILTER (WHERE shipping_cost < 0) AS shipping_cos
t_negativos,
    COUNT(*) FILTER (WHERE order_year NOT BETWEEN 1000 AND 99
99) AS order_year_invalido
FROM transformation.orders;

-- 8. Validar rango de campos numéricos
SELECT
    MIN(sales) AS sales_min,
    MAX(sales) AS sales_max,
    MIN(quantity) AS quantity_min,
    MAX(quantity) AS quantity_max,
    MIN(discount) AS discount_min,
    MAX(discount) AS discount_max,
    MIN(profit) AS profit_min,
    MAX(profit) AS profit_max,
    MIN(shipping_cost) AS shipping_cost_min,
    MAX(shipping_cost) AS shipping_cost_max,
    MIN(order_year) AS order_year_min,
    MAX(order_year) AS order_year_max
FROM transformation.orders;

```

```

-- 9. Validar cantidad máxima de decimales en campos NUMERIC
SELECT
    MAX(
        CASE
            WHEN discount IS NULL THEN 0
            WHEN POSITION('.') IN discount::TEXT = 0 THEN 0
            ELSE LENGTH(SPLIT_PART(discount::TEXT, '.', 2))
        END
    ) AS max_decimales_discount,
    MAX(
        CASE
            WHEN profit IS NULL THEN 0
            WHEN POSITION('.') IN profit::TEXT = 0 THEN 0
            ELSE LENGTH(SPLIT_PART(profit::TEXT, '.', 2))
        END
    ) AS max_decimales_profit,
    MAX(
        CASE
            WHEN shipping_cost IS NULL THEN 0
            WHEN POSITION('.') IN shipping_cost::TEXT = 0 THEN 0
            ELSE LENGTH(SPLIT_PART(shipping_cost::TEXT, '.', 2))
        END
    ) AS max_decimales_shipping_cost
FROM transformation.orders;

-- 10. Validar dígitos máximos antes del punto decimal
SELECT
    MAX(LENGTH(ABS(sales)::TEXT)) AS max_digitos_sales,
    MAX(LENGTH(ABS(quantity)::TEXT)) AS max_digitos_quantity,
    MAX(LENGTH(SPLIT_PART(ABS(discount)::TEXT, '.', 1))) AS max_digitos_enteros_discount,

```

```

        MAX(LENGTH(SPLIT_PART(ABS(profit)::TEXT, '.', 1))) AS max
        _digitos_enteros_profit,
        MAX(LENGTH(SPLIT_PART(ABS(shipping_cost)::TEXT, '.', 1)))
AS max_digitos_enteros_shipping_cost,
        MAX(LENGTH(ABS(order_year)::TEXT)) AS max_digitos_order_y
ear
FROM transformation.orders;

```

-- 11. Validar longitud máxima de campos de texto

```

SELECT
    MAX(LENGTH(order_id)) AS order_id_max,
    MAX(LENGTH(ship_mode)) AS ship_mode_max,
    MAX(LENGTH(customer_name)) AS customer_name_max,
    MAX(LENGTH(segment)) AS segment_max,
    MAX(LENGTH(state_name)) AS state_name_max,
    MAX(LENGTH(country)) AS country_max,
    MAX(LENGTH(market)) AS market_max,
    MAX(LENGTH(region)) AS region_max,
    MAX(LENGTH(product_id)) AS product_id_max,
    MAX(LENGTH(category)) AS category_max,
    MAX(LENGTH(sub_category)) AS sub_category_max,
    MAX(LENGTH(product_name)) AS product_name_max,
    MAX(LENGTH(order_priority)) AS order_priority_max
FROM transformation.orders;

```

-- 12. Validar coherencia de fechas

```

SELECT COUNT(*) AS envios_antes_de_orden
FROM transformation.orders
WHERE ship_date < order_date;

```

-- 13. Validar coherencia entre order_date y order_year

```

SELECT COUNT(*) AS inconsistencias_order_year

```

```
FROM transformation.orders
WHERE EXTRACT(YEAR FROM order_date)::INTEGER <> order_year;
```

Consultas de validación final en **analytics**

-- 1. Validar que analytics.orders no tenga campos nulos

```
SELECT COUNT(*) AS registros_con_nulos
```

```
FROM analytics.orders
```

```
WHERE record_id IS NULL
```

```
OR order_id IS NULL
```

```
OR order_date IS NULL
```

```
OR ship_date IS NULL
```

```
OR ship_mode IS NULL
```

```
OR customer_name IS NULL
```

```
OR segment IS NULL
```

```
OR state_name IS NULL
```

```
OR country IS NULL
```

```
OR market IS NULL
```

```
OR region IS NULL
```

```
OR product_id IS NULL
```

```
OR category IS NULL
```

```
OR sub_category IS NULL
```

```
OR product_name IS NULL
```

```
OR sales IS NULL
```

```
OR quantity IS NULL
```

```
OR discount IS NULL
```

```
OR profit IS NULL
```

```
OR shipping_cost IS NULL
```

```
OR order_priority IS NULL
```

```
OR order_year IS NULL;
```

-- 2. Validar que record_id sea único

```
SELECT
```

```
COUNT(*) AS total_record_id,
```

```

COUNT(DISTINCT record_id) AS record_id_unicos
FROM analytics.orders;

-- 3. Validar reglas numéricas finales
SELECT
    COUNT(*) FILTER (WHERE sales < 0) AS sales_negativos,
    COUNT(*) FILTER (WHERE quantity <= 0) AS quantity_cero_o_
negativa,
    COUNT(*) FILTER (WHERE discount < 0 OR discount > 1) AS d
iscount_fuera_rango,
    COUNT(*) FILTER (WHERE shipping_cost < 0) AS shipping_cos
t_negativos,
    COUNT(*) FILTER (WHERE order_year NOT BETWEEN 1000 AND 99
99) AS order_year_invalido
FROM analytics.orders;

-- 4. Ver muestra final de datos analíticos
SELECT *
FROM analytics.orders
LIMIT 10;

-- 5. Consulta analítica básica: ventas por año
SELECT
    order_year,
    COUNT(*) AS total_registros,
    SUM(sales) AS total_sales,
    SUM(quantity) AS total_quantity,
    SUM(profit) AS total_profit
FROM analytics.orders
GROUP BY order_year
ORDER BY order_year;

-- 6. Consulta analítica básica: ventas por categoría

```

```
SELECT
    category,
    COUNT(*) AS total_registros,
    SUM(sales) AS total_sales,
    SUM(quantity) AS total_quantity,
    SUM(profit) AS total_profit
FROM analytics.orders
GROUP BY category
ORDER BY total_sales DESC;
```

-- 7. Consulta analítica básica: ventas por región

```
SELECT
    region,
    COUNT(*) AS total_registros,
    SUM(sales) AS total_sales,
    SUM(quantity) AS total_quantity,
    SUM(profit) AS total_profit
FROM analytics.orders
GROUP BY region
ORDER BY total_sales DESC;
```